

Study Buddy

"Study Buddy" is an embodied quadruped robot that utilizes computer vision and speech recognition to monitor a user's focus during study sessions and responds with real-time expressive behaviors. An evaluation of the system demonstrated that users spent more time on-task and experienced a greater sense of accountability when studying with the interactive robot compared to a standard countdown timer.

Team Member: Lele Zhao, Role: Speech Recognition module ([Section 3.1.2](#))

Team Member: Juan Yin, Role: Computer Vision module ([Section 3.1.1](#))

Team Member: Nicole Go, Role: Workflow and Behavior module ([Section 3.1.3](#))



1. Executive Summary

Study Buddy is an embodied robot designed to support focus during Pomodoro study sessions. The robot combines computer vision, speech interaction, and expressive behaviors to create a real-time study companion that can monitor user activity and provide feedback throughout a session.

The system consists of three main components. A computer vision module detects user presence and phone usage using an onboard OAK-D camera. A behavior controller implemented as a finite-state machine combines these signals with a Pomodoro timer to manage the robot's emotional and session states. A speech interface allows users to control study sessions and manage tasks through voice commands. Together, these components create a complete sense–express–intervene loop that enables the robot to respond to changes in user behavior in real time.

We evaluated the system using a within-subjects study in which participants completed identical study tasks under two conditions: a plain countdown timer and the full Study Buddy system. Results show that participants spent more time on-task, reported higher engagement, and expressed greater willingness to reuse the system in the Study Buddy condition. Qualitative feedback further suggests that the robot's continuous expressive presence and timely reactions contributed to a stronger sense of accountability attention, compared to the non-social baseline.

Overall, the findings indicate that even minimal real-time social embodiment can meaningfully influence study behavior and user experience. This work demonstrates a lightweight design approach for embodied companion systems and provides a platform for further investigation into long-term effects of social presence on self-regulated study behavior.

2. Introduction

Maintaining focus during long study sessions is challenging. Students are frequently distracted by their phones, and studying alone for extended periods can feel isolating. While existing tools such as timers, app blockers, and focus apps can help manage distractions, they primarily rely on self-discipline and provide little social engagement.

Our goal was to explore whether a robot companion could provide a more engaging approach to focus support. We developed Study Buddy, a quadruped robot whose wellbeing is linked to the user's study behavior. Instead of relying entirely on willpower, the system encourages focus through companionship and caretaking. The robot responds to user actions through movement, facial expressions, sounds, and other social behaviors, creating the feeling of studying alongside a companion rather than simply using a timer.

This design is motivated by findings from human-robot interaction research. Prior work has shown that physically embodied robots can create stronger engagement than screen-based systems and that people often attribute emotions and intentions to lifelike behaviors [1, 3, 4, 5]. Research has also demonstrated that expressive companion robots can encourage long-term interaction and behavior change through social attachment and caretaking [6–9]. We apply these ideas to the context of solo studying.

Study Buddy creates a complete sense–express–intervene loop. Using computer vision and speech input, the robot monitors user presence, phone usage, and study session progress. It then responds through expressive behaviors and timer-based interactions designed to encourage continued focus.

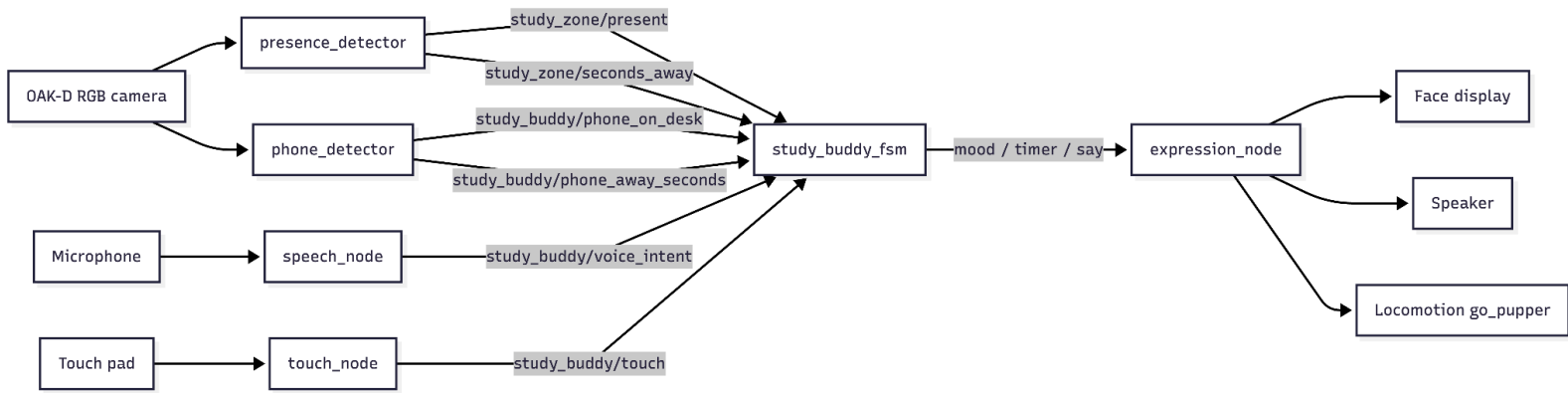
This project investigates two research questions: (1) Can an embodied study companion reduce off-task phone use during studying? (2) Can it increase feelings of companionship and engagement compared to studying alone? The

remainder of this report describes the technical system, evaluation methodology, and lessons learned from developing Study Buddy.

3.1 Technical approach

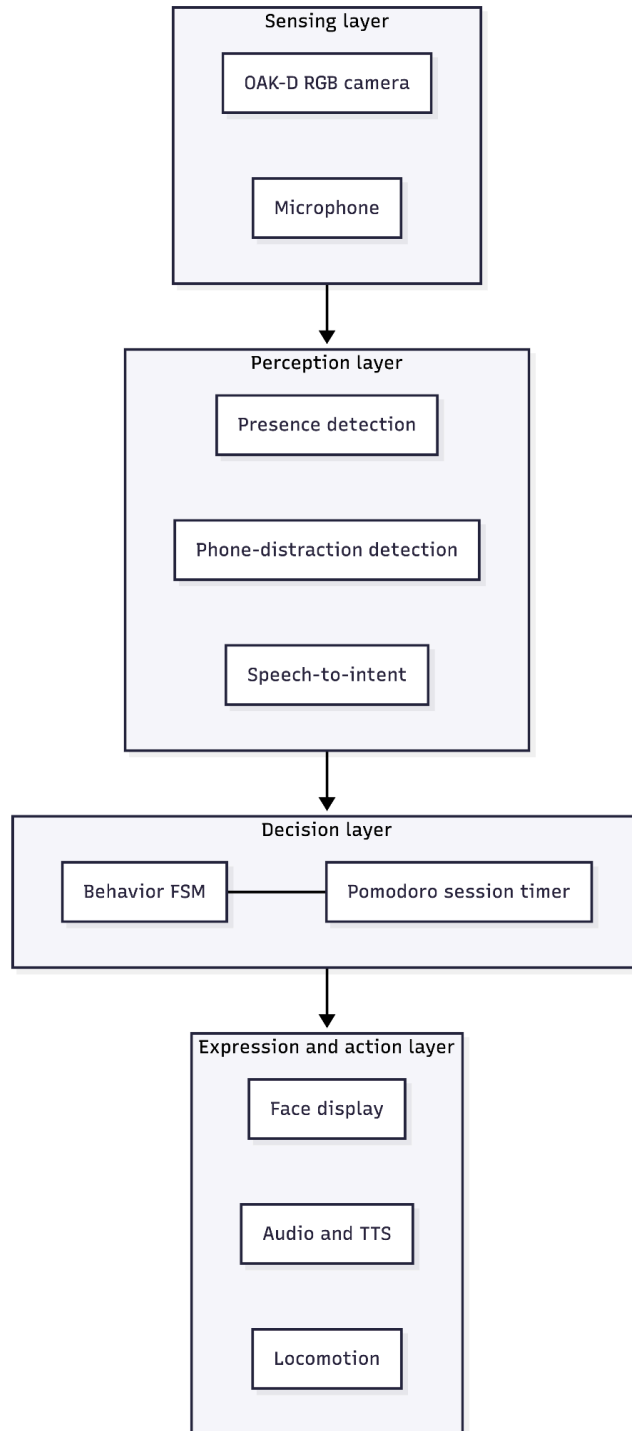
Study Buddy is built as a collection of ROS 2 nodes that communicate through named topics. The system is divided into three main components: computer vision, speech, and behavior control. Two computer vision nodes detect user presence and phone activity, a speech node converts spoken commands into intents, and a touch node detects interactions with the robot. These signals are sent to a centralized behavior finite-state machine (FSM), which manages the study session and controls the robot's facial expressions, audio responses, and movement.

This modular architecture allows each component to be developed and tested independently. It also improves system reliability, since failures in one node do not affect the rest of the system. The separation was particularly useful for our three person team, allowing each member to work on a different subsystem while communicating through predefined ROS topics. Figure 1 shows the high-level software architecture.



[Figure 1. High-level software architecture. Computer vision, speech, and touch nodes publish perceptual events to the behavior FSM, which controls the robot's expressive outputs.]

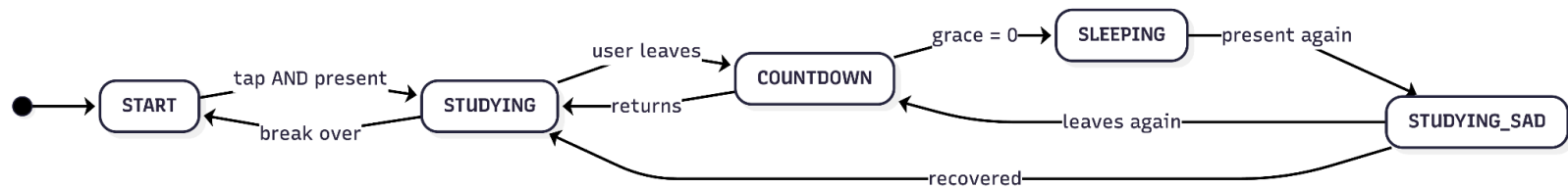
The overall system follows a sense, perceive, decide, express loop. Sensors, including the camera, microphone, and touch pad, collect raw input. The perception layer converts this input into a small set of meaningful signals, such as user presence, phone status, and voice commands. These signals are processed by the FSM, which maintains the robot's internal state, session timers, and decision logic. Finally, the expression layer communicates the robot's state through facial expressions, audio feedback, and movement behaviors. Figure 2 illustrates this control architecture.



[Figure 2. Overall control architecture. Sensor data is processed into perceptual events, interpreted by the FSM, and expressed through the robot's behaviors.]

The robot's behavior is governed by a finite-state machine that maps user actions and timer events to emotional and study-session states. Presence events determine transitions such as studying, waiting, sleeping, and resuming, while phone and voice events trigger engagement-related responses. Centralizing these decisions in the FSM keeps the

perception modules lightweight and ensures consistent behavior across the system. Figure 3 presents the complete stimulus–response diagram, and the following sections describe each subsystem in detail.



[Figure 3. Study Buddy stimulus–response diagram. Presence, phone, and voice events drive transitions between the robot's study-session and emotional states.]

3.1.1 Technical Sub Task 1: Computer Vision Node (Lead: Juan Yin)

Goal and Design Requirements

The perception layer answers two questions in real time: whether the user is present at the desk, and whether the phone is on the desk or being picked up. Each is published as a smoothed binary signal used by the behavior FSM.

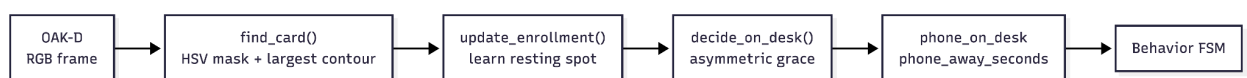
The main constraint is compute. Everything runs on a Raspberry Pi alongside speech recognition and behavior control, so the system must be lightweight, stable, and robust to noise. This leads to three requirements: low CPU cost, resistance to false triggers, and isolated components that can fail without crashing the system.

Pipeline Architecture

Both detectors use the same camera stream, frame downsampling, and throttling strategy. They subscribe to the OAK-D RGB feed (/oak/rgb/image_raw) and convert images using cv_bridge. The system runs in RGB-only mode because depth and onboard inference caused instability on our hardware.

Each node processes 320px-width frames and runs inference every N frames. We also use qos_profile_sensor_data (best-effort, depth 1), so when the system is overloaded, old frames are dropped instead of queued, preventing lag buildup.

Phone-Distraction Detection



[Figure 4 - phone_detection pipeline]

Since direct phone detection is too heavy for the Pi, we use a neon marker as a proxy. Detection uses a simple CV pipeline: `cv2.inRange()` (HSV filtering), `cv2.findContours()`, and `cv2.moments()` for centroid tracking. The pipeline shows in Figure 4.

The system first locks the phone's resting position via `update_enrollment()` after it stays stable for `REST_STABLE_SEC` within `MOVE_PX`. Only then does movement count as a pickup, preventing false triggers.

Decision logic in `decide_on_desk()` is asymmetric: longer delay for pickup (`PICKUP_GRACE`) and shorter delay for recovery (`RESUME_GRACE`), making the system stable but responsive.

A `calibrate()` function auto-generates HSV bounds for different lighting conditions.

Presence Detection



[Figure 5 - human_detection pipeline]

Presence is a binary “user in frame” decision with a swappable backend via a shared `detect()` interface. The pipeline shows in Figure 5.

Primary backend uses MediaPipe Pose (`model_complexity = 0`) and detects presence when at least two torso landmarks (shoulders and hips) exceed confidence thresholds. A HOG-based OpenCV detector serves as fallback.

A short smoothing window (`AWAY_SECONDS = 2.0`) filters frame noise while preserving real leave/return transitions.

Integration with the FSM

Both detectors are intentionally minimal and only publish smoothed signals. The presence node outputs `study_zone/present` (Bool) and `study_zone/seconds_away` (Float), while the phone node outputs `study_buddy/phone_on_desk` (Bool) and `study_buddy/phone_away_seconds` (Float). The behavior FSM consumes all four signals.

Presence drives session-level transitions (leave, return, idle), while phone state drives fine-grained engagement responses (focus vs. distraction). This separation keeps perception lightweight while delegating temporal reasoning and behavioral policy to the FSM.

This architecture is what makes the system stable under constrained compute: short-term noise is handled locally in perception, while higher-level interpretation is centralized in the FSM.

3.1.2 Technical Sub Task 2: Speech Recognition Node (Lead: Lele Zhao)

Goal and Design Requirements



[Figure 6 - Speech node pipeline]

The speech recognition node gives Study Buddy hands-free voice control: the user speaks a command such as "pause," "resume," or "add to-do item," and the robot updates the study session accordingly. We imposed three design requirements on this module. First, all processing must run fully on-device, with no cloud services, to preserve user privacy and to keep the system functional without network access. Second, the module must capture only intentional commands rather than continuously listening, both to respect the user and to avoid transcribing ambient room noise. Third, the module must operate within the limited compute budget of the Raspberry Pi while the camera, perception, and behavior nodes run concurrently.

Pipeline Architecture

The module is structured as a linear pipeline: microphone capture, push-to-talk triggering, local speech-to-text transcription, intent parsing, and publication to the ROS graph (Figure 6). Each stage lives in its own Python module — `recorder.py` (audio capture and triggers), `asr.py` (the transcription wrapper), `intents.py` (text-to-intent mapping), and `tts.py` (spoken feedback) — with a thin ROS node, `speech_node.py`, wiring them together and a single `config.py` holding all tunable parameters. This separation keeps the intent logic a pure function with no hardware dependency, which allowed us to unit-test command recognition on a development laptop without a microphone or robot.

The node communicates with the rest of the system exclusively through ROS 2 topics. After each utterance, it publishes a small JSON intent message on `study_buddy/voice_intent`; the behavior FSM subscribes to this topic and owns all session-state changes. The speech module therefore never touches the timer or display directly, which establishes a clean contract between modules and means the heavy transcription stage could be relocated off-board without changing any consumer of the topic.

Push-to-Talk Trigger

We implemented a hold-to-talk trigger using the Pupper's built-in capacitive front touch pad: the user holds the pad, speaks, and releases, and the node records exactly that interval. This design choice directly improved recognition quality. In early testing with a free-running trigger, every capture ran to a 15-second safety cap and fed the recognizer long windows of ambient noise, producing slow and inaccurate transcriptions; hold-to-talk instead yields short, clean utterances of one to two seconds that transcribe roughly an order of magnitude faster and far more reliably.

Reading the touch pad required solving a hardware-ownership conflict. The pad's GPIO line (BCM 6) is claimed at boot by the kernel's gpio-keys driver, so the conventional gpiozero interface, which requests exclusive ownership of the line through the GPIO character device, fails with a "GPIO busy" error. We diagnosed this by inspecting line consumers with gpioinfo and confirming that no userspace process held the line. Our solution reads the pin's hardware register directly through the RPi.GPIO library with active-low polling, the same access pattern used by the platform's reference touch controller, which coexists with the kernel driver. The trigger is selectable in configuration among three modes — terminal Enter key for desktop development, an external GPIO button, and the built-in touch pad — which supported incremental testing across development machines and the robot.

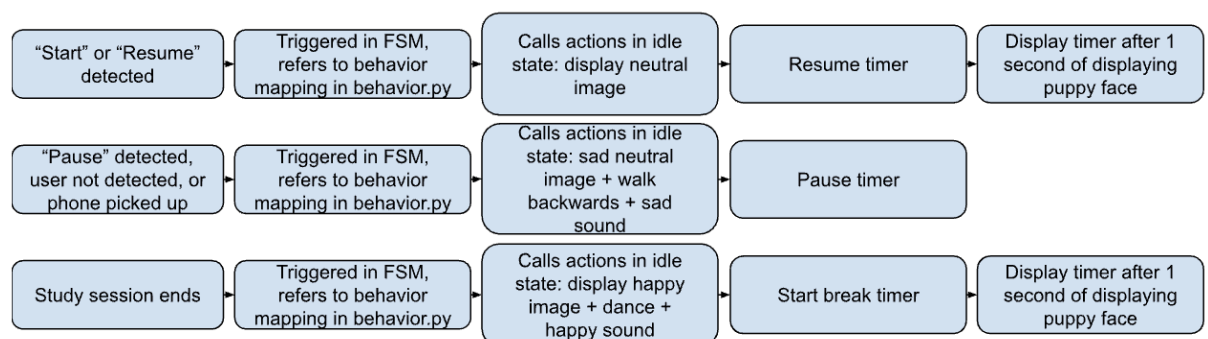
On-Device Transcription and Intent Parsing

Transcription uses faster-whisper running the tiny.en Whisper model with int8 quantization, the configuration that fit the Pi's compute envelope while remaining accurate on short English commands. Because Whisper claims all CPU cores by default and starved the concurrent camera and perception nodes, we capped the recognizer at two of the Pi's four cores via the library's cpu_threads parameter, trading a modest increase in per-utterance latency for stable operation of the full stack.

The transcribed text is mapped to one of four intents — PAUSE, RESUME, ADD_TODO, or UNKNOWN — by a keyword-based parser that matches each intent against a set of natural phrasings (for example, "stop," "wait," and "hold on" all map to PAUSE). On a RESUME intent the node additionally publishes a wake message, and on an UNKNOWN intent the robot speaks "I didn't get that" through an on-board espeak text-to-speech path, closing the interaction loop so the user always receives feedback.

3.1.3 Technical Sub Task 3: Workflow and Behavior (Lead: Nicole Go)

Goal and Design Requirements



[Figure 7 - Behavior pipeline]

The workflow and behavior aspects allow the Study Buddy to react to voice commands from the speech recognition node and actions detected by the computer vision node. It contributes to the gamification of the user's study experience. Its purpose is to react happily to the user studying and become a sad pet-like buddy when the user is distracted. Requirements to create this effective

user experience included a fast enough reaction time, a functional study and break timer, and 2-3 actions occurring simultaneously in response to a trigger to exude that emotion.

Pipeline Architecture



[Figure 8 - neutral, sad, happy states]

The pipeline has the same process but can go one of three reactions depending on the voice command and action. The general workflow is that the voice command or action sends a trigger, which is received in the FSM file. The FSM file then refers to `behavior.py`, which has a mapping for the emotions that defines the respective display, sound, and movement for that emotion. It also refers to `pomodoro.py`. Then, each of those actions is called simultaneously. If the user says “start” or “resume”, the Study Buddy enters an idle state so it simply displays its neutral image. At the same time, the study timer either starts and displays after 1 second. If the user says “pause”, is detected to be holding their phone, or is no longer detected as present, then the Study Buddy enters a sad state in which it displays a sad image, plays a sad sound, and moves backwards. The timer also gets paused and stays on the sad face. Once the study session ends, the Study Buddy celebrates and enters a happy state, meaning it displays a happy face, plays a happy sound, and dances. After 1 second, the break timer displays. Additionally, the displays have dog food in the corner. The dog is most fed when he is happy, has no food when he is sad, and is half full at neutral.

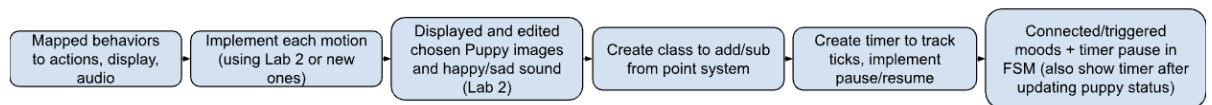
Display, Audio, and Movement

To display the images, play the audios, and perform the movements, we referred to the Lab 2 and Lab 2 Optional Challenges. For display and audio, we used the `Display` and `sounddevice` packages as well as converted them to the right size and binary format. For movements, we referred to the interfaces given to us in Lab 2. In the mapping, there are randomized choices between the movements. For instance, being happy could randomly make the Study Buddy walk forward or dance so that the varied movements make it more natural. We also implemented some new actions such as crouching.

Pomodoro Timer

Our pomodoro timer uses the Time package and has functions to start, pause, and resume. Along with maintaining the timer, the function keeps track of the state the Study Buddy is in and adjusts flags that inform the FSM if it is paused or resumed. It also keeps track of the amount of ticks that have passed to inform the other files on how long studying or the lack of studying has been happening.

Integration into FSM



[Figure 9 - Behavior timeline]

The triggers, behavior, and timer are all managed in the FSM. It started as mapping the behaviors, then implementing the timer, then connecting them and the triggers in the FSM. Once the FSM senses a trigger, it refers to the `refresh_mood` function, which sets the state and declares what mood the Study Buddy will express on the behavior map. It then counts the ticks for 1 second to pass to create a buffer before referring to the step function, which continuously tracks every tick. After 1 second, it calls to use an imported draw function to display text representing how many minutes are left. It then repeats this cycle of study sessions and breaks until the user is done.

3.2 Methodology

Overview

We evaluated Study Buddy, an expressive and socially reactive study-companion robot, in a within-subjects, mixed-methods experiment designed to test whether the robot's social presence and reactivity improve user focus and study experience relative to a non-social baseline. The experiment manipulated a single independent variable, the form of the study companion, with two levels: a plain countdown timer (control condition) and the complete Study Buddy system (treatment condition), which includes expressive mood displays, presence-aware reactions, phone-distraction detection, and voice-based session control. We tested two hypotheses. H1 states that participants spend a greater proportion of the session on-task in the treatment condition. H2 states that participants report a more engaging and motivating study experience in the treatment condition.

We selected a within-subjects design so that each participant experiences both conditions and serves as their own control. This design is well suited to small samples because it eliminates between-subject variability in baseline study habits, which is a dominant source of noise in focus-related measures, and it increases sensitivity for a given number of participants. The primary threats to internal validity in this design are ordering, practice, and novelty effects, which we address through counterbalancing and matched study materials, as described in the Protocol subsection. Consistent with standard practice in human-robot interaction, we combined validated questionnaire instruments, behavioral measures logged by the system, and qualitative interview data.

Before recruiting participants, we piloted and iterated on the study itself. We prepared a paper study checklist enumerating every step of a session, including the materials to bring and the scripts to read aloud, and ran a complete pilot session with a friend acting as a participant. Based on their feedback we revised the orientation script and checklist, then ran a second pilot with a different friend to confirm the revised protocol ran smoothly before beginning actual data collection.

Participants

Six participants ($N = 6$) took part in the study, recruited via convenience sampling from our university community. All participants were master's students in the Computer Science and Engineering department at UC San Diego, aged [21–24]. None of the participants had prior exposure to the Study Buddy system, and none was affiliated with the project team, which avoids demand characteristics arising from familiarity with our hypotheses. Participation was voluntary and uncompensated, and participants provided informed consent before the session began.

Each participant was assigned an anonymized identifier (P1 through P6) at the start of their session. All data files, transcripts, and analyses refer to participants only by these identifiers; the key linking identifiers to individuals was kept on paper, offline, and was not shared.

Protocol

Each session lasted approximately 45 minutes and was conducted in a quiet room with the participant seated at a desk. The session followed six phases. First, we welcomed the participant, completed the informed-consent process, and invited questions. Second, we gave a brief overview of what would happen during the session. Third, the participant completed a short practice period with the robot before the main study began: they tapped the touch pads, tried the "pause" and "resume" voice commands, and saw each facial expression along with an explanation of its meaning, following a standardized orientation script that was identical for all participants. This practice phase ensured consistent prior knowledge across sessions and reduced novelty effects during measurement.

Fourth, the participant completed the main study: two 15-minute self-directed study blocks, one per condition, separated by a short break. In the control condition, the participant studied with only a plain countdown timer visible; all expressive, reactive, and voice features were disabled. In the treatment condition, the full system was active: the robot displayed the remaining session time, reacted to the participant's presence and phone use, expressed moods through its face and motion, and responded to spoken "pause" and "resume" commands. To control for content effects, the two blocks used study materials of comparable difficulty but distinct content, preventing carryover of answers between blocks. Condition order was counterbalanced: three participants completed the control condition first and three completed the treatment condition first, with order assigned randomly. Counterbalancing distributes ordering, practice, fatigue, and novelty effects across both conditions rather than confounding either one.

To mitigate the Hawthorne effect, the experimenter started each block and then left the room, leaving the participant alone with the system for the duration of the block. This was possible because all behavioral

measures are logged automatically by the robot's own sensing pipeline, so no human observation was required during the blocks themselves.

Fifth, after each block the participant completed a questionnaire evaluating that condition, ensuring ratings were collected while the experience was recent and before exposure to the other condition could anchor responses. After both blocks, we conducted a brief semi-structured exit interview in person and verbally, since participants generally share more when speaking than when typing into forms. The interview used neutral, non-leading prompts, such as "How was your experience studying with the robot?", "Was there anything that helped or hindered your focus?", and "Is there anything you would change?" Finally, we thanked the participant for their time.

Measures

We operationalized focus through objective behavioral measures logged automatically by the system's own sensing pipeline, which removes experimenter judgment from the measurement process, ensures identical measurement across conditions, and enabled the unobserved study blocks described above. The presence detector recorded time on-task and cumulative time away from the study zone; the phone-use detector recorded the number of phone pickups and total phone-use duration. We additionally recorded block completion (whether the participant finished the block without abandoning it) and administered a short comprehension quiz on the study material after each block as a secondary measure of study effectiveness. The quiz for each block was matched in length and question format.

Subjective experience was measured with the Likability subscale of the Godspeed questionnaire [2], a validated and widely used instrument in human-robot interaction research drawn from the HRI scale database, administered after each condition. We supplemented Godspeed with four custom items rated on 7-point Likert scales, chosen to target our constructs of interest directly: "I felt accountable to the robot," "I stayed focused during this session," "I would use this system to study," and "The system was distracting" (reverse-coded). The verbal exit interviews provided qualitative data, which we analyzed for recurring themes such as perceived accountability and sources of distraction.

With the participants' consent, exit interviews were audio-recorded for later transcription. We collected the minimum data necessary: only audio was recorded, recordings were stored in a password-protected location under anonymized filenames (e.g., P2_interview), and all recordings are deleted after the conclusion of the project. No personal identifiers appear in any data file or in this report.

Given the sample size, our analysis is primarily descriptive: we report each participant's values in both conditions and the within-participant direction of change on every measure, alongside thematic findings from the interviews. A consistent direction of change across most or all participants on a measure constitutes converging directional support for the corresponding hypothesis, and the qualitative data serve to explain the mechanisms behind any observed differences.

4 Results

Quantitative Results

	P1	P2	P3	P4	P5	P6
On Task (%)	71	70	85	65	70	80
Phone Pickups	2	4	1	2	1	2

[Figure 10 - On Task Percentage and Phone Pickups for Control]

	P1	P2	P3	P4	P5	P6
On Task (%)	91	84	93	86	75	80
Phone Pickups	1	2	0	0	1	2

[Figure 11 - On Task Percentage and Phone Pickups for Treatment]

Throughout their tasks for the control and treatment, we observed how often they would look at their work screen to focus on their task and how productive they were. We also observed how often they picked up their phone. The on-task percentage increased by at least 10% for 3 of the participants and at least 5% for 2 of them. The remaining participant didn't have a change in focus. The number of phone pickups decreased for 4 of the participants.

Participant / Likert Scale Question	I felt accountable to the robot	I stayed focused during this session	I would use this system to study	The system was distracting
P1	Agree	Agree	Neutral	Disagree
P2	Strongly Agree	Agree	Agree	Disagree
P3	Agree	Neutral	Agree	Disagree
P4	Agree	Agree	Agree	Neutral
P5	Neutral	Agree	Neutral	Disagree
P6	Disagree	Agree	Disagree	Agree

[Figure 12 - Likert scale Questions Table]

Out of the 6 participants, 4 either agreed or strongly agreed that they felt accountable for the robot. The others either felt neutral or disagreed. For staying focused during the session, 5 felt they were and the remaining felt neutral. When asked if they would use the Study Buddy, half agreed they would while 2

were neutral, and another disagreed. Finally, 4 out of 6 felt it wasn't distracting while one was neutral and the last agreed it was distracting. Overall, there is a correlation between feeling accountable for the robot and feeling more inclined to study. It worked better for those who felt it wasn't distracting. Ultimately, it could be up to personal preference on if they wanted an alarming-like Study Buddy like ours to focus.

Qualitative Results

Within our interviews, most participants reported a positive experience with the Study Buddy due to feeling encouraged when the robot would celebrate and feeling down when the robot was sad. There was a sense of feeling accountable to maintain the Study Buddy's wellbeing, which could have only been done by staying on task. They felt less of an urge to find a reason to leave. One participant claimed that "it felt like they were being watched" and another said "I just wanted to keep the robot happy so it made me want to stay focused".

When asked if anything hindered their focus, the main concern was the audio of the Study Buddy as it was hard to hear. This was expressed by 4 of the participants. One participant reported that they could tell what the Study Buddy was trying to exclaim due to its portrayed mood and movements, but couldn't hear as well as these other aspects. Conversely, another user struggled with those visual cues, noting they couldn't figure out the difference between the Study Buddy's neutral and happy expressions. Some participants had concerns about how pressing the front pin and saying commands to take a break might feel too strict on if the user has to leave. Another participant expressed that the bright colored paper on their phone made them look at their phone even more. Finally, another participant brought up that they didn't prefer the Study Buddy because they found the concept and movements distracting.

When asking if they'd change anything or add features, some participants expressed interest in the point system idea and claimed it would be helpful to see the points on the screen because it would add to the gamification aspect. They had similar suggestions to the previous question on making the audio louder. One suggested using a QR code instead of bright paper. Finally, one suggested having a way to show the recommended commands that work better with our speech recognition system such as informing the user that "continue" works better than "resume".

Overall, our quantitative results support our H1 and our qualitative results support our H2 in about two-thirds to three-fourths of our participants. This is because most of them reported a higher percentage of being focused in the treatment group and because most of them reported positive experiences of feeling engaged. It also seems this method for studying might work for some people who might prefer working under pressure, while being distracting to others who prefer quietness.

5 Discussion and Future Work

Looking ahead, Study Buddy can now be seen as a complete, real-time loop running entirely on a desk. It can detect basic study states: when the user leaves, returns, puts the phone down, or picks it up, and translate those signals into different behaviors. The robot leans into the idea of "shared attention": it approaches when focus drops, and perks up when the user comes back. This already lets us directly test

the core question behind the project: whether a visible, embodied companion changes off-task behavior more than self-control alone. It also opens the door to studying companionship itself, since the robot maintains a continuous expressive presence instead of only reacting at failure points.

A key lesson from building the system is that perception design is tightly constrained by on-device compute. Early versions included richer components like gaze tracking and learned phone detection, but running them together with speech recognition and behavior control quickly overwhelmed the Raspberry Pi and caused instability. The final system worked because we simplified aggressively: reducing perception to a small number of binary signals, throttling and downscaling processing, using frame dropping, and prioritizing safe degradation over completeness. We also leaned toward “positive evidence only” detection, so uncertainty would not trigger incorrect feedback. In practice, this trade-off mattered more than model sophistication, because a single wrong “you’re distracted” can easily break trust in a companion robot.

Future work naturally follows from the limitations. The most direct improvement is replacing the color-marker approach with an on-device object detector for phones, ideally offloaded to dedicated hardware like a Coral TPU or an OAK-D, which would also unlock more robust perception pipelines such as depth-based “study zone” tracking. Reintroducing lightweight gaze estimation on specialized compute could further refine intent detection, distinguishing active studying from idle looking. Beyond perception, the bigger open question is long-term interaction: whether the emotional effect of Study Buddy persists, fades, or evolves over weeks of use. A longitudinal study would be necessary to answer this. Finally, the system could become more adaptive by personalizing response timing and Pomodoro-style intervals to match each user’s study style over time.

REFERENCES

- [1] J. Wainer, D. J. Feil-Seifer, D. A. Shell, and M. J. Matarić, "The role of physical embodiment in human–robot interaction," in Proc. IEEE Int. Symp. Robot and Human Interactive Communication, 2006:117–122.
- [2] Bartneck, C. (2023). Godspeed Questionnaire Series: Translations and Usage. In C. U. Krägeloh, O. N. Medvedev, & M. Alyami (Eds.), International Handbook of Behavioral Health Assessment (pp. 1-35). Springer. https://doi.org/10.1007/978-3-030-89738-3_24-1
- [3] K. M. Lee, Y. Jung, J. Kim, and S. R. Kim, "Are physically embodied social agents better than disembodied social agents? The effects of physical embodiment, tactile interaction, and people's loneliness in human–robot interaction," Int. J. Human-Computer Studies, vol. 64, no. 10, 2006.

- [4] Bartneck C, Belpaeme T, Eyssel F, Kanda T, Keijsers M, Šabanović S. How People Perceive Robots. In Human-Robot Interaction: An Introduction. Cambridge University Press; 2024:134-147.
- [5] J. Forlizzi and C. DiSalvo, "Service robots in the domestic environment: A study of the Roomba vacuum in the home," in *Proc. ACM/IEEE HRI*, 2006:258–265.
- [6] Bartneck C, Belpaeme T, Eyssel F, Kanda T, Keijsers M, Šabanović S. Nonverbal Interaction. In: Human-Robot Interaction: An Introduction. Cambridge University Press; 2020:81-97.
- [7] C. D. Kidd and C. Breazeal, "Robots at home: Understanding long-term human–robot interaction," in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2008:3230–3235.
- [8] I. Leite, C. Martinho, and A. Paiva, "Social robots for long-term interaction: A survey," Int. J. Social Robotics, vol. 5, no. 2, 2013:291–308.
- [9] F. Kaplan, "Free creatures: The role of uselessness in the design of artificial pets," in Proc. 1st Edutainment Robotics Workshop, 2000.
- [9] Hoffman, G., & Zhao, X. (2020). A primer for conducting experiments in human–robot interaction. ACM Transactions on Human–Robot Interaction, 10(1), Article 6. <https://doi.org/10.1145/3412374>
- [10] Louise Veling, & Conor McGinn (2021). Qualitative research in HRI: A review and taxonomy. Springer Nature.
- [11] Goodrich, M. A., & Schultz, A. C. (2007). Human–robot interaction: A survey. Foundations and Trends in Human–Computer Interaction, 1(3), 203–275. <https://doi.org/10.1561/11000000005>